

```
# Importando bibliotecas
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import BernoulliNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
# Lendo o csv
df = pd.read_csv('selecionados_novo.csv', sep=';')
df.head()
```

	var_0	var_1	var_2	var_3	var_4	var_5	var_9	var_11	var_14	var_16	var_23
0	0	4	15	24	1	0	0	1	3	0	0
1	0	4	15	23	0	0	4	0	1	0	1
2	1	4	21	23	0	0	4	2	3	0	1
3	0	4	14	22	1	0	4	2	1	0	0
4	1	4	15	14	0	0	4	1	3	0	1

```
# Criando um dataframe somente com as features
X = df.drop('var_0',axis=1)
```

```
# Criando um dataframe somente com as classes
y = df['var_0']
```

```
# Definindo as métricas de avaliação
scoring = ['accuracy', 'precision','recall', 'f1', 'roc_auc']
```

```
# Variaveis para armazenar o melhor classificador de cada modelo
per_best_estimator = ""
per_best_score = 0
knn_best_estimator = ""
knn_best_score = 0
nb_best_estimator = ""
nb_best_score = 0
nb_distribution = ""
dt_best_estimator = ""
```

```

dt_best_score = 0
rf_best_estimator = ""
rf_best_score = 0
mlp_best_estimator = ""
mlp_best_score = 0
svc_best_estimator = ""
svc_best_score = 0
svc_kernel = ""

```

▼ PERCEPTRON

```

# Instanciando o classificador
per = Perceptron()

# Definindo alguns parametros para teste
params = {
    'penalty': ['l2','l1','elasticnet']
}

# Instanciando o gridsearch e executando o modelo

for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=per,
                               param_grid=params,
                               cv = 4,
                               n_jobs=-1, verbose=1, scoring=score_item)

    #%%time
    grid_search.fit(X, y)
    best = grid_search.best_estimator_
    print('Melhor classificador: ',best)
    print('Score: ',grid_search.best_score_)
    print()
    if (per_best_score < grid_search.best_score_):
        per_best_estimator = best
        per_best_score = grid_search.best_score_

Métrica de avaliacao:  accuracy
Fitting 4 folds for each of 3 candidates, totalling 12 fits
Melhor classificador:  Perceptron(penalty='l1')
Score:  0.858695652173913

Métrica de avaliacao:  precision
Fitting 4 folds for each of 3 candidates, totalling 12 fits
Melhor classificador:  Perceptron(penalty='l1')
Score:  0.6666666666666666

Métrica de avaliacao:  recall
Fitting 4 folds for each of 3 candidates, totalling 12 fits
Melhor classificador:  Perceptron(penalty='l2')
Score:  0.4375

```

```
Métrica de avaliacao: f1
Fitting 4 folds for each of 3 candidates, totalling 12 fits
Melhor classificador: Perceptron(penalty='elasticnet')
Score: 0.39166666666666666
```

```
Métrica de avaliacao: roc_auc
Fitting 4 folds for each of 3 candidates, totalling 12 fits
Melhor classificador: Perceptron(penalty='l1')
Score: 0.8914473684210527
```

▼ KNN

```
# Instanciando o classificador
knn = KNeighborsClassifier(n_jobs=-1)

# Definindo alguns parametros para teste
params = {
    'n_neighbors': [2,3,5,10,20],
    'metric': ['euclidean','manhattan','chebyshev']
}

# Instanciando o gridsearch e executando o modelo

for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=knn,
                               param_grid=params,
                               cv = 4,
                               n_jobs=-1, verbose=1, scoring=score_item)

    %%time
    grid_search.fit(X, y)
    best = grid_search.best_estimator_
    print('Melhor classificador: ',best)
    print('Score: ',grid_search.best_score_)
    print()
    if (knn_best_score < grid_search.best_score_):
        knn_best_estimator = best
        knn_best_score = grid_search.best_score_

Métrica de avaliacao: accuracy
Fitting 4 folds for each of 15 candidates, totalling 60 fits
Melhor classificador: KNeighborsClassifier(metric='euclidean', n_jobs=-1, n_neighbo
Score: 0.8586956521739131

Métrica de avaliacao: precision
Fitting 4 folds for each of 15 candidates, totalling 60 fits
Melhor classificador: KNeighborsClassifier(metric='euclidean', n_jobs=-1)
Score: 0.75

Métrica de avaliacao: recall
```

```
Fitting 4 folds for each of 15 candidates, totalling 60 fits
Melhor classificador: KNeighborsClassifier(metric='euclidean', n_jobs=-1, n_neighbo
Score: 0.375
```

```
Métrica de avaliacao: f1
Fitting 4 folds for each of 15 candidates, totalling 60 fits
Melhor classificador: KNeighborsClassifier(metric='manhattan', n_jobs=-1, n_neighbo
Score: 0.4720238095238095
```

```
Métrica de avaliacao: roc_auc
Fitting 4 folds for each of 15 candidates, totalling 60 fits
Melhor classificador: KNeighborsClassifier(metric='manhattan', n_jobs=-1, n_neighbo
Score: 0.830592105263158
```



▼ Naive Bayes

```
nb = GaussianNB()
params = {'var_smoothing': np.logspace(0,-9, num=10)}
for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=nb,
                               param_grid=params,
                               cv = 4,
                               n_jobs=-1, verbose=1, scoring=score_item)

    #%%time
    grid_search.fit(X, y)
    best = grid_search.best_estimator_
    print('Melhor classificador: ',best)
    print('Score: ',grid_search.best_score_)
    print()
    if (nb_best_score < grid_search.best_score_):
        nb_best_estimator = best
        nb_best_score = grid_search.best_score_
        nb_distribution = 'Gaussian'
```

```
Métrica de avaliacao: accuracy
Fitting 4 folds for each of 10 candidates, totalling 40 fits
Melhor classificador: GaussianNB(var_smoothing=1.0)
Score: 0.8369565217391304
```

```
Métrica de avaliacao: precision
Fitting 4 folds for each of 10 candidates, totalling 40 fits
Melhor classificador: GaussianNB(var_smoothing=0.1)
Score: 0.6805555555555556
```

```
Métrica de avaliacao: recall
Fitting 4 folds for each of 10 candidates, totalling 40 fits
Melhor classificador: GaussianNB(var_smoothing=0.0001)
Score: 0.375
```

```
Métrica de avaliacao: f1
Fitting 4 folds for each of 10 candidates, totalling 40 fits
Melhor classificador: GaussianNB(var_smoothing=0.1)
```

Score: 0.36025641025641025

Métrica de avaliacao: roc_auc
Fitting 4 folds for each of 10 candidates, totalling 40 fits
Melhor classificador: GaussianNB(var_smoothing=0.001)
Score: 0.8157894736842106

```
nb = MultinomialNB()
params = {
    'alpha': [0.1,0.2,0.3,0.5,0.7,1.0]
}
for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=nb,
                               param_grid=params,
                               cv = 4,
                               n_jobs=-1, verbose=1, scoring=score_item)

    %%time
    grid_search.fit(X, y)
    best = grid_search.best_estimator_
    print('Melhor classificador: ',best)
    print('Score: ',grid_search.best_score_)
    print()
    if (nb_best_score < grid_search.best_score_):
        nb_best_estimator = best
        nb_best_score = grid_search.best_score_
        nb_distribution = 'Multinomial'
```

Métrica de avaliacao: accuracy
Fitting 4 folds for each of 6 candidates, totalling 24 fits
Melhor classificador: MultinomialNB(alpha=0.1)
Score: 0.782608695652174

Métrica de avaliacao: precision
Fitting 4 folds for each of 6 candidates, totalling 24 fits
Melhor classificador: MultinomialNB(alpha=0.1)
Score: 0.19642857142857142

Métrica de avaliacao: recall
Fitting 4 folds for each of 6 candidates, totalling 24 fits
Melhor classificador: MultinomialNB(alpha=0.1)
Score: 0.1875

Métrica de avaliacao: f1
Fitting 4 folds for each of 6 candidates, totalling 24 fits
Melhor classificador: MultinomialNB(alpha=0.1)
Score: 0.17424242424242425

Métrica de avaliacao: roc_auc
Fitting 4 folds for each of 6 candidates, totalling 24 fits
Melhor classificador: MultinomialNB(alpha=0.1)
Score: 0.7894736842105263

```
nb = BernoulliNB()
params = {
```

```

params = {
    'alpha': [0.1,0.2,0.3,0.5,0.7,1.0]
}
for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=nb,
                               param_grid=params,
                               cv = 4,
                               n_jobs=-1, verbose=1, scoring=score_item)

    #%%time
    grid_search.fit(X, y)
    best = grid_search.best_estimator_
    print('Melhor classificador: ',best)
    print('Score: ',grid_search.best_score_)
    print()
    if (nb_best_score < grid_search.best_score_):
        nb_best_estimator = best
        nb_best_score = grid_search.best_score_
        nb_distribution = 'Bernoulli'

    Métrica de avaliacao:  accuracy
    Fitting 4 folds for each of 6 candidates, totalling 24 fits
    Melhor classificador:  BernoulliNB(alpha=0.1)
    Score:  0.8043478260869565

    Métrica de avaliacao:  precision
    Fitting 4 folds for each of 6 candidates, totalling 24 fits
    Melhor classificador:  BernoulliNB(alpha=0.3)
    Score:  0.375

    Métrica de avaliacao:  recall
    Fitting 4 folds for each of 6 candidates, totalling 24 fits
    Melhor classificador:  BernoulliNB(alpha=0.1)
    Score:  0.1875

    Métrica de avaliacao:  f1
    Fitting 4 folds for each of 6 candidates, totalling 24 fits
    Melhor classificador:  BernoulliNB(alpha=0.1)
    Score:  0.23809523809523808

    Métrica de avaliacao:  roc_auc
    Fitting 4 folds for each of 6 candidates, totalling 24 fits
    Melhor classificador:  BernoulliNB()
    Score:  0.7286184210526316

```

▼ Decision Tree

```

dt = DecisionTreeClassifier(random_state=42)
params = {
    'criterion': ['gini','entropy'],
    'max_depth': [2,3,5,10,20],
    'min_samples_leaf': [5,10,20,50,100,200]
}
for score_item in scoring:

```

```

print('Métrica de avaliacao: ', score_item)
grid_search = GridSearchCV(estimator=dt,
                           param_grid=params,
                           cv = 4,
                           n_jobs=-1, verbose=1, scoring=score_item)

#%%time
grid_search.fit(X, y)
best = grid_search.best_estimator_
print('Melhor classificador: ',best)
print('Score: ',grid_search.best_score_)
print()
if (dt_best_score < grid_search.best_score_):
    dt_best_estimator = best
    dt_best_score = grid_search.best_score_

Métrica de avaliacao:  accuracy
Fitting 4 folds for each of 60 candidates, totalling 240 fits
Melhor classificador:  DecisionTreeClassifier(max_depth=5, min_samples_leaf=5, random_state=42)
Score:  0.8369565217391305

Métrica de avaliacao:  precision
Fitting 4 folds for each of 60 candidates, totalling 240 fits
Melhor classificador:  DecisionTreeClassifier(max_depth=2, min_samples_leaf=5, random_state=42)
Score:  0.75

Métrica de avaliacao:  recall
Fitting 4 folds for each of 60 candidates, totalling 240 fits
Melhor classificador:  DecisionTreeClassifier(criterion='entropy', max_depth=5, min_samples_leaf=5, random_state=42)
Score:  0.4375

Métrica de avaliacao:  f1
Fitting 4 folds for each of 60 candidates, totalling 240 fits
Melhor classificador:  DecisionTreeClassifier(max_depth=5, min_samples_leaf=5, random_state=42)
Score:  0.4361111111111111

Métrica de avaliacao:  roc_auc
Fitting 4 folds for each of 60 candidates, totalling 240 fits
Melhor classificador:  DecisionTreeClassifier(max_depth=3, min_samples_leaf=10, random_state=42)
Score:  0.743421052631579

```



▼ Random Forest

```

rf = RandomForestClassifier(random_state=42)
params = {
    'criterion': ['gini','entropy'],
    'max_depth': [2,3,5,10,20],
    'min_samples_leaf': [5,10,20,50,100,200],
    'n_estimators': [10,25,30,50,100,200]
}
for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=rf,

```

```

        param_grid=params,
        cv = 4,
        n_jobs=-1, verbose=1, scoring=score_item)

#%%time
grid_search.fit(X, y)
best = grid_search.best_estimator_
print('Melhor classificador: ',best)
print('Score: ',grid_search.best_score_)
print()
if (rf_best_score < grid_search.best_score_):
    rf_best_estimator = best
    rf_best_score = grid_search.best_score_

Métrica de avaliacao: accuracy
Fitting 4 folds for each of 360 candidates, totalling 1440 fits
Melhor classificador: RandomForestClassifier(max_depth=2, min_samples_leaf=5, n_estimators=100,
                                             random_state=42)
Score: 0.8369565217391304

Métrica de avaliacao: precision
Fitting 4 folds for each of 360 candidates, totalling 1440 fits
Melhor classificador: RandomForestClassifier(max_depth=2, min_samples_leaf=5, n_estimators=100,
                                             random_state=42)
Score: 0.25

Métrica de avaliacao: recall
Fitting 4 folds for each of 360 candidates, totalling 1440 fits
Melhor classificador: RandomForestClassifier(max_depth=2, min_samples_leaf=5, n_estimators=100,
                                             random_state=42)
Score: 0.0625

Métrica de avaliacao: f1
Fitting 4 folds for each of 360 candidates, totalling 1440 fits
Melhor classificador: RandomForestClassifier(max_depth=2, min_samples_leaf=5, n_estimators=100,
                                             random_state=42)
Score: 0.1

Métrica de avaliacao: roc_auc
Fitting 4 folds for each of 360 candidates, totalling 1440 fits
Melhor classificador: RandomForestClassifier(max_depth=2, min_samples_leaf=10, n_estimators=100,
                                             random_state=42)
Score: 0.8289473684210527

```



▼ MLP

```

mlp = MLPClassifier(max_iter=100)
params = {
    'hidden_layer_sizes': [(10,30,10),(20,)],
    'activation': ['tanh', 'relu'],
    'solver': ['sgd', 'adam'],
    'alpha': [0.0001, 0.05],
    'learning_rate': ['constant', 'adaptive'],
}

```

```

for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=mlp,
                               param_grid=params,
                               cv = 4,
                               n_jobs=-1, verbose=1, scoring=score_item)

    %%time
    grid_search.fit(X, y)
    best = grid_search.best_estimator_
    print('Melhor classificador: ',best)
    print('Score: ',grid_search.best_score_)
    print()
    if (mlp_best_score < grid_search.best_score_):
        mlp_best_estimator = best
        mlp_best_score = grid_search.best_score_

Métrica de avaliacao:  accuracy
Fitting 4 folds for each of 32 candidates, totalling 128 fits
/usr/local/lib/python3.7/dist-packages/sklearn/neural_network/_multilayer_perceptron
ConvergenceWarning,
Melhor classificador:  MLPClassifier(hidden_layer_sizes=(10, 30, 10), max_iter=100)
Score:  0.8804347826086956

Métrica de avaliacao:  precision
Fitting 4 folds for each of 32 candidates, totalling 128 fits
/usr/local/lib/python3.7/dist-packages/sklearn/neural_network/_multilayer_perceptron
ConvergenceWarning,
Melhor classificador:  MLPClassifier(hidden_layer_sizes=(10, 30, 10), learning_rate=
max_iter=100)
Score:  0.9375

Métrica de avaliacao:  recall
Fitting 4 folds for each of 32 candidates, totalling 128 fits
/usr/local/lib/python3.7/dist-packages/sklearn/neural_network/_multilayer_perceptron
ConvergenceWarning,
Melhor classificador:  MLPClassifier(activation='tanh', alpha=0.05, hidden_layer_siz
learning_rate='adaptive', max_iter=100)
Score:  0.5625

Métrica de avaliacao:  f1
Fitting 4 folds for each of 32 candidates, totalling 128 fits
/usr/local/lib/python3.7/dist-packages/sklearn/neural_network/_multilayer_perceptron
ConvergenceWarning,
Melhor classificador:  MLPClassifier(hidden_layer_sizes=(10, 30, 10), max_iter=100)
Score:  0.6595238095238095

Métrica de avaliacao:  roc_auc
Fitting 4 folds for each of 32 candidates, totalling 128 fits
Melhor classificador:  MLPClassifier(activation='tanh', hidden_layer_sizes=(10, 30,
learning_rate='adaptive', max_iter=100)
Score:  0.861842105263158

/usr/local/lib/python3.7/dist-packages/sklearn/neural_network/_multilayer_perceptron
ConvergenceWarning,

```

▼ SVM

```
#kernel rbf
svc_kernel = 'rbf'
svc = SVC(kernel='rbf')
params = {'C': [0.1, 1, 10, 100],
          'gamma': [1,0.1, 0.01, 0.001]}
for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=svc,
                               param_grid=params,
                               cv = 4,
                               n_jobs=-1, verbose=1, scoring=score_item)

    #%%time
    grid_search.fit(X, y)
    best = grid_search.best_estimator_
    print('Melhor classificador: ',best)
    print('Score: ',grid_search.best_score_)
    print()
    if (svc_best_score < grid_search.best_score_):
        svc_best_estimator = best
        svc_best_score = grid_search.best_score_
```

Métrica de avaliacao: accuracy
Fitting 4 folds for each of 16 candidates, totalling 64 fits
Melhor classificador: SVC(C=100, gamma=0.001)
Score: 0.8913043478260869

Métrica de avaliacao: precision
Fitting 4 folds for each of 16 candidates, totalling 64 fits
Melhor classificador: SVC(C=10, gamma=0.001)
Score: 0.9375

Métrica de avaliacao: recall
Fitting 4 folds for each of 16 candidates, totalling 64 fits
Melhor classificador: SVC(C=10, gamma=0.01)
Score: 0.5625

Métrica de avaliacao: f1
Fitting 4 folds for each of 16 candidates, totalling 64 fits
Melhor classificador: SVC(C=100, gamma=0.001)
Score: 0.6279761904761905

Métrica de avaliacao: roc_auc
Fitting 4 folds for each of 16 candidates, totalling 64 fits
Melhor classificador: SVC(C=100, gamma=0.001)
Score: 0.8552631578947368

```
#kernel linear
svc_kernel = 'linear'
svc = SVC(kernel='linear')
params = {'C': [0.1,1,10]}
for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=svc,
```

```

grid_search = GridSearchCV(estimator=svc,
                           param_grid=params,
                           cv = 4,
                           n_jobs=-1, verbose=1, scoring=score_item)

#%%time
grid_search.fit(X, y)
best = grid_search.best_estimator_
print('Melhor classificador: ',best)
print('Score: ',grid_search.best_score_)
print()
if (svc_best_score < grid_search.best_score_):
    svc_best_estimator = best
    svc_best_score = grid_search.best_score_

```

Métrica de avaliacao: accuracy
 Fitting 4 folds for each of 3 candidates, totalling 12 fits
 Melhor classificador: SVC(C=0.1, kernel='linear')
 Score: 0.9130434782608695

Métrica de avaliacao: precision
 Fitting 4 folds for each of 3 candidates, totalling 12 fits
 Melhor classificador: SVC(C=0.1, kernel='linear')
 Score: 0.9375

Métrica de avaliacao: recall
 Fitting 4 folds for each of 3 candidates, totalling 12 fits
 Melhor classificador: SVC(C=1, kernel='linear')
 Score: 0.6875

Métrica de avaliacao: f1
 Fitting 4 folds for each of 3 candidates, totalling 12 fits
 Melhor classificador: SVC(C=1, kernel='linear')
 Score: 0.7023809523809524

Métrica de avaliacao: roc_auc
 Fitting 4 folds for each of 3 candidates, totalling 12 fits
 Melhor classificador: SVC(C=0.1, kernel='linear')
 Score: 0.9144736842105263

```

#kernel polinomial
svc_kernel = 'polinomial'
svc = SVC(kernel='poly')
params = {'C': [0.1, 1, 10],
          'gamma': [1,0.1,0.01],
          'degree': [3,4]}
for score_item in scoring:
    print('Métrica de avaliacao: ', score_item)
    grid_search = GridSearchCV(estimator=svc,
                              param_grid=params,
                              cv = 4,
                              n_jobs=-1, verbose=1, scoring=score_item)

#%%time
grid_search.fit(X, y)
best = grid_search.best_estimator_
print('Melhor classificador: ',best)
print('Score: ',grid_search.best_score_)

```

```
print()
if (svc_best_score < grid_search.best_score_):
    svc_best_estimator = best
    svc_best_score = grid_search.best_score_
```

Métrica de avaliacao: accuracy
Fitting 4 folds for each of 18 candidates, totalling 72 fits
Melhor classificador: SVC(C=0.1, degree=4, gamma=0.01, kernel='poly')
Score: 0.8804347826086957

Métrica de avaliacao: precision
Fitting 4 folds for each of 18 candidates, totalling 72 fits
Melhor classificador: SVC(C=0.1, degree=4, gamma=0.01, kernel='poly')
Score: 0.7083333333333333

Métrica de avaliacao: recall
Fitting 4 folds for each of 18 candidates, totalling 72 fits
Melhor classificador: SVC(C=0.1, degree=4, gamma=0.01, kernel='poly')
Score: 0.625

Métrica de avaliacao: f1
Fitting 4 folds for each of 18 candidates, totalling 72 fits
Melhor classificador: SVC(C=0.1, degree=4, gamma=0.01, kernel='poly')
Score: 0.65

Métrica de avaliacao: roc_auc
Fitting 4 folds for each of 18 candidates, totalling 72 fits
Melhor classificador: SVC(C=1, gamma=0.01, kernel='poly')
Score: 0.8256578947368423